SIGma **Fall 2026, October 22th, 2026**

2027 CG:SHOP Background

Lecturer: Ian Chen *Scribe: Ian Chen*

1 prelim

Note: the following text is a mixture of what I said, what I think I said, and what I wish I said. As such, there may be small discrepancies from the lecture. Of course, there may be errors, and there is no peer-review.

This semester, we are preparing a submission for CG:SHOP [2], part of the annual Symposium on Computational Geometry (SoCG). The problem for this year is *multi-robot lawn mowing*:

- **input**: axis-aligned polygonal regions R (lawn) and χ (cutter), and a parameter k
- **output**: axis-aligned tours $\tau_1, \ldots, \tau_k$ such that $\bigcup_{i \in [\![k]\!]} \bigcup_{p_j \in \tau_i} \chi(p_j) \supseteq R$, minimizing $\max_{i \in [\![k]\!]} |\tau_i|$

Here, I am using the notation $[\![k]\!] = \{1, \ldots, k\}$, $\chi(p)$ to denote placing the cutter centered at point p. All regions are in the plane $\mathcal{R}^2$. For this problem in particular, each edge of a region has integer side lengths, and so do the output tours (I will refer to this as integrality). See the 2nd meeting (`www.cstheory.org/meetings/`) for a slides/video description.

Now, this is a difficult problem to try and reason about. For example, some of the challenges include:

- an infinite amount of potential tours and different regions; moreover without integrality, it is uncountable!
- arbitrary regions R and χ that can be non-convex or be any other nasty shapes
- combinatorial explosion from planning k simultaneous tours

Therefore, for the first few weeks, we shall focus on some special cases which are a bit easier to think about. For example, we could:

- only consider one robot at a time (avoiding the combinatorial explosion)
- or restrict the shape of the cutter χ or the region

or there are many other simplifications, as we will later see. For today (and next week), we will restrict χ to be a unit square and consider only $k = 1$ (one robot).

2 one-square lawn mowing

This is still a very difficult problem, but it is also very geometric: given some subset of the plane[1], plan a path covering it. Thus, naturally one can come up with a suite of intuitive covering/routing plans. In Fig. 1, I show three such plans, walking around boundaries, or taking horizontal/vertical sweeps. What concerns us now is to be able to evaluate such algorithms.

What would such an objective function look like? Some properties that we could want are[2]:

- interpretable (the values of this objective have meaning)
- comparable (the value is a good proxy that does not depend on a specific algorithm)

[1] In this section, we will not assume that all side lengths are integral.

[2] Johnson [3] is a great paper on experimental algorithms that I keep referring to.

- generalizable (the value does not depend only on a specific set of examples)
- amenable to mathematic calculations

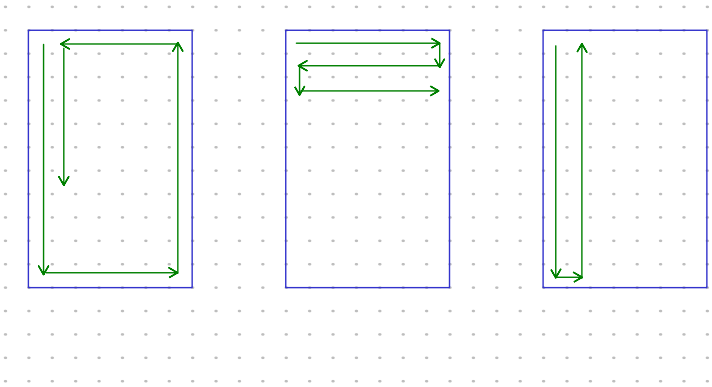


Figure 1: **Natural heuristics:** Left: a tour that walks around the boundaries. Center: a tour that sweeps horizontally. Right: a tour that sweeps vertically.

For example, let $\mathcal{A}$ be an algorithm for the one-square lawn mowing problem, i.e. $\mathcal{A}$ takes in a region R and outputs a tour. The first idea proposed in the lecture is to use the formula in Eq. 1.

$$\frac{|\mathcal{A}(R)|}{|R|} \tag{1}$$

This is good because we can compare different algorithms according to it – if algorithm $\mathcal{A}_1$ has a lower value than $\mathcal{A}_2$, then $\mathcal{A}_1$ is probably better. But it is not generalizable, as it depends on a set of regions R to calculate it on. It may be interpreted as the ratio of our algorithm to some "idealistic" algorithm that visits each square in the region exactly once. This is fine, and is fairly easy to calculate mathematically.

Let us try to improve the generalizability. In analysis of algorithms, a typical approach is to consider the worst-case, as in Eq. 2. However, if we try to calculate it, we have a problem.

$$\max_R \frac{|\mathcal{A}(R)|}{|R|} \tag{2}$$

Consider for example a region R that is an $\epsilon \times 2$ rectangle. Then, any algorithm must have length at least 2, but the area is 2ϵ. Therefore, for any algorithm, this quantity is at least $1/\epsilon$; as ϵ decreases this quantity becomes larger and larger. In this somewhat straightforward approach for generalizability, Eq. 2 degenerates to always being ∞.

The problem is that there is a lack of distinction between hard and easy to cover regions. The previous paragraph gave us a hint on what to do: normalize instead by the best achievable value by any algorithm.

$$\max_R \frac{|\mathcal{A}(R)|}{\min_{\mathcal{A}'} |\mathcal{A}'(R)|} \tag{3}$$

Eq. 3 is known as the *approximation ratio* for an algorithm $\mathcal{A}$. It is generalizable (not being specific to any specific instances) and comparable. Moreover, it is interpretable; a value of 2 means that on any instance, the length of the tour is at most double the best achievable one. We shall use Eq. 3 as the metric from now on.

> **Exercise 2.1:** Consider the algorithm $\mathcal{A}$ that walks around the boundary of the polygon, as in the left of Fig. 1. It turns out that the approximation ratio also tends towards ∞, i.e. for any constant c, there are regions that it costs more than c times the optimal. Prove this.

How can we design an algorithm that has a good approximation ratio? The approach taken in [1] is to notice that lawn mowing (which is to find a path covering the region) is very similar to the Traveling Salesman problem (TSP), of finding a path visiting a set of nodes. We will describe this reduction in more detail next week; for now, we define the TSP and sketch an approximation algorithm for it.

2.1 an approx algo for traveling salesman

Consider a set of n points in $\mathcal{R}^2$. The TSP problem is to find a path that visits each point exactly once, minimizing the length of the path. For example, given the geographic locations of 48 state capitals, plan a trip visiting each of them in the shortest possible time. Indeed, this feels familiar to the lawn mowing problem.

This problem is also difficult to solve exactly, and there are well-known approximation algorithms. Namely, it turns out that one can find the *minimum spanning tree*, i.e. an acyclic subgraph of edges that connects all the points, minimizing the sum of distances in the edges. Intuitively, finding paths is very constrained in a way that is very non-local, while trees relax this structure making optimization easier.

Somehow, this tree looks to be close to what we want. What's left is to be able to turn this tree into a path.

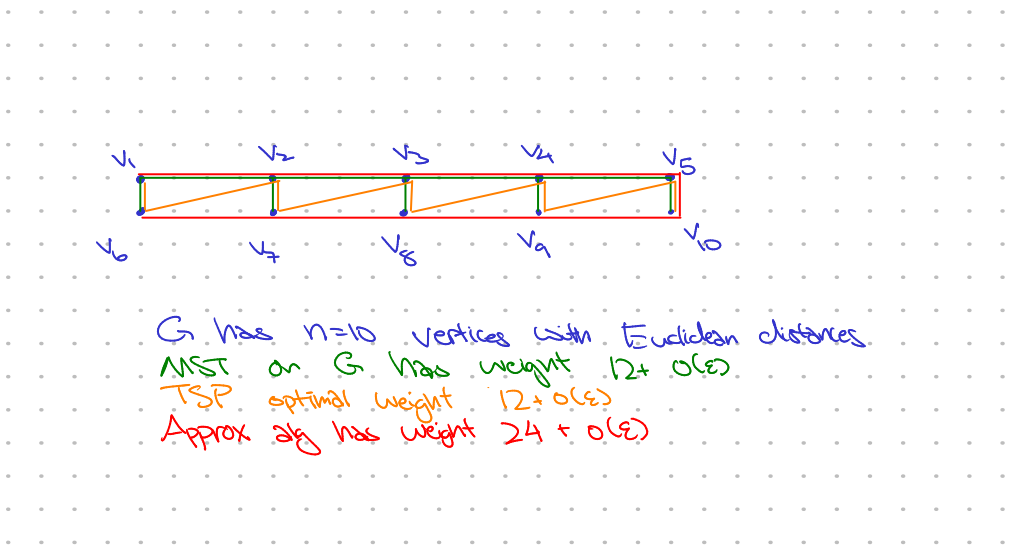


Figure 2: **MST-based TSP approx. algorithm:** Blue: vertices $v_1, \ldots, v_{10}$ in $\mathcal{R}^2$ plane. $d(v_i, v_{i+5}) = \epsilon$ for $i \leq 5$ and $d(v_i, v_{i+1}) = 3$ for $i \neq 5$. Green: Minimum spanning tree on vertices is a "comb" shape. Orange: Optimal traveling salesman path is a "zig-zag" shape. Red: output of the approximation algorithm; see text.

In Fig. 2, we show an example on a graph with 10 vertices. The MST (green) is a comb-like shape, with edges (v_1, v_2), (v_2, v_3), (v_3, v_4), (v_4, v_5), (v_1, v_6), (v_2, v_7), (v_3, v_8), (v_4, v_9), (v_5, v_{10}). We can walk around the tree as $v_1v_2v_3v_4v_5v_{10}v_5v_4v_9v_4v_3v_8v_3v_2v_7v_2v_1v_6$ to visit each vertex. This is not a path (as we visit a vertex multiple times); this is easy to fix, by removing duplicate vertices, obtaining the path shown as red in Fig. 2: $v_1v_2v_3v_4v_5v_{10}v_9v_8v_6v_6$.

It is easy to bound the approximation ratio. The initial walk around the tree at most doubles the length (as we use each edge at most twice). Then, removing vertices takes shortcuts, and does not increase the length. As the initial tree is no more than the optimal TSP weight (by the property of the minimum spanning tree), then

$$2 * \text{opt TSP} \geq 2 * \text{MST} \geq \text{path} \implies \text{path/opt TSP} \leq 2$$

solutions

> **Solution to Exercise 2.1:** Consider a "dumbbell" region R_k, consisting of two $k \times k$ squares separated by a path of length k^2. The boundary-walking algorithm outputs a tour of length $\approx k^3$, despite the optimal being of length $\approx 4k^2$. As k increases, the approximation is unbounded.

References

[1] Esther M. Arkin, Sándor P. Fekete, and Joseph S.B. Mitchell. "Approximation algorithms for lawn mowing and milling". In: *Computational Geometry* 17.1 (2000), pp. 25–50. ISSN: 0925-7721. DOI: https://doi.org/10.1016/S0925-7721(00)00015-8.

[2] *CG:SHOP: Computational Geometry: Solving Hard Optimization Problems*. URL: https://cgshop.ibr.cs.tu-bs.de/ (visited on 10/23/2026).

[3] David S Johnson. *Experimental analysis of algorithms*. Vol. 59. American Mathematical Society, Providence, RI, USA, 2002.